

The GUAC Shop – Creating a Cloud Datawarehouse and Retail Analytics

*Prepared by: **Sagar Naduvinkeri***

This report outlines the design and implementation of a data warehouse for The Guac Stop (TGS), a retail store chain specializing in locally sourced avocados and avocado-themed merchandise. In the earlier phase of this project, we conducted a comprehensive business requirements gathering exercise to understand the company's operational challenges and data requirements. We discovered that TGS lacked a centralized data repository, making it difficult to generate actionable insights from their sales data and manage vendor, employee, and customer information efficiently.

From the initial business requirements analysis, we identified the key dimensions and facts necessary for TGS to improve its data-driven decision-making capabilities. Specifically, we established the following dimensions:

1. Customer
2. Product
3. Store
4. Vendor
5. Date

These dimensions were designed to support a central fact table:

- Sales

With this foundational understanding, we moved forward to build the data warehouse using Oracle Cloud Infrastructure (OCI).

Key Steps in Data Warehouse Implementation:

1. **Creation of Dimension Tables:** Each dimension table (Customer, Product, Vendor, Store and Date) was created to store descriptive data that would be used to analyze and slice sales data in various ways.
2. **Creation of the Fact Table:** The Sales fact table is at the core of the data warehouse. It stores transactional data such as product sales, quantities sold, total prices, and ties back to each dimension through foreign keys.

3. **Establishing Relationships Between Fact and Dimension Tables:** Relationships between the fact and dimension tables were established to ensure seamless query performance and easy navigation between different data points. This forms the basis of a Star Schema.
4. **Designing the Star Schema with Oracle Data Modeler:** We used Oracle's Data Modeler tool to design the database architecture in a star schema. The star schema design simplifies complex queries and enhances the performance of aggregating large datasets, which is critical for retail data warehouses like TGS's, where slicing data by store, vendor, and product is essential for reporting.

Enterprise Bus Matrix:

Business Process	Common Dimensions				
	Customer	Product	Vendor	Store	Date
Sales Reporting	✖	✖	✖	✖	✖
Vendor Management			✖		
Customer Insights	✖	✖			✖
Employee Performance Tracking				✖	
Product performance by Season		✖			✖
Promotions and Discounts	✖	✖			✖
Online vs Offline Sales		✖		✖	✖

Stakeholder Interview:

1. How do you currently measure sales outcome?

This Question helped understand the existing Key performance Indicators (KPIs) and the sales metrics in use.

2. What are your future goals?

This Question helped to align the data warehouse design with long-term business goals such as growth and expansion.

3. Where do you feel you are lacking visibility today?

This Question helped understand the current pain points of the business, missing data and in gap in current reporting.

4. What current tools and applications do you use?

This Question helped to assess the existing technology stack and evaluate which systems need to be integrated with the warehouse.

5. What is missing in your sales data report?

This question helped to identify the key gaps such as customer, product, vendor information, that need to be addressed.

6. Can you differentiate between Online and Offline Sales today?

This question helped to identify whether the sales data is tracked by channel (Online vs Offline) and improve visibility across channels.

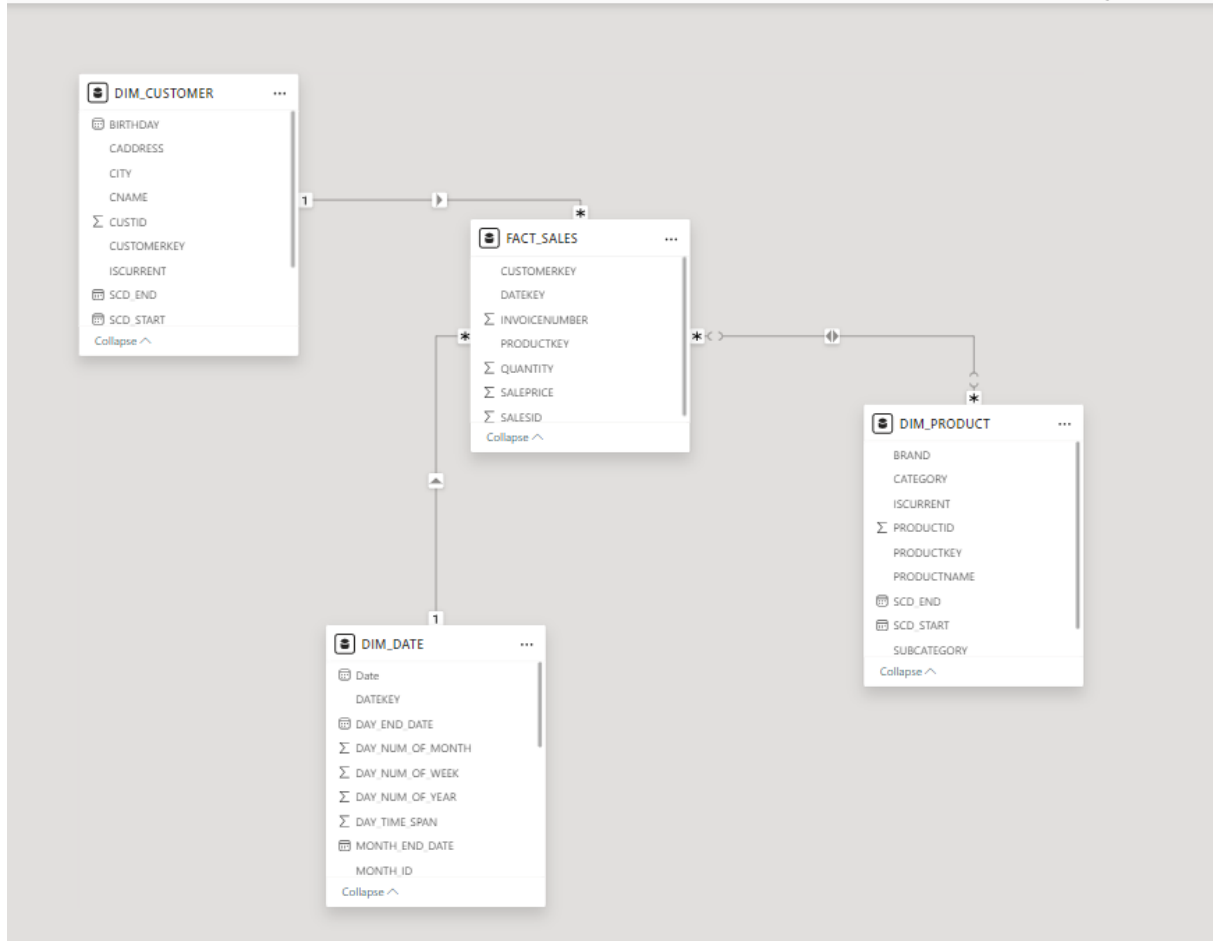
7. Do you track employee performance and vendor performance?

This Question helped to identify the missing information on the employee performance and vendor data.

8. Do you track product sales over seasonality?

This Question helped to identify the granularity of the product data being captured currently and to increase it in the warehouse design.

Facts and Dimension tables:



Customer Dimension Creation:

```
CREATE TABLE ADMIN.DIM_CUSTOMER
(
    CUSTOMER_ID      NUMBER GENERATED ALWAYS AS IDENTITY
        ( START WITH 1 CACHE 20 ) ,
    CUSTOMER_NAME    VARCHAR2 (128) ,
    CONTACT_NUMBER   NUMBER ,
    ADDRESS           VARCHAR2 (20) ,
    LOYALTY_STATUS   VARCHAR2 (20)
)
TABLESPACE DATA
LOGGING
;

CREATE UNIQUE INDEX ADMIN.CUSTOMER_PK ON ADMIN.DIM_CUSTOMER
(
    CUSTOMER_ID ASC
)
TABLESPACE DATA
LOGGING
;

ALTER TABLE ADMIN.DIM_CUSTOMER
    ADD CONSTRAINT CUSTOMER_PK PRIMARY KEY ( CUSTOMER_ID )
    USING INDEX ADMIN.CUSTOMER_PK ;

ALTER TABLE ADMIN.DIM_CUSTOMER
    ADD CONSTRAINT CUST_TO_PROD FOREIGN KEY
    (
        CUSTOMER_ID
    )
    REFERENCES ADMIN.DIM_CUSTOMER ( CUSTOMER_ID )
    ON DELETE CASCADE
    NOT DEFERRABLE NOVALIDATE
```

Product Dimension Creation:

```
CREATE TABLE ADMIN.DIM_PRODUCT
(
    PRODUCT_ID    NUMBER GENERATED ALWAYS AS IDENTITY
                  ( START WITH 1 CACHE 20 ) ,
    PRODUCT_NAME  VARCHAR2 (128) ,
    CATEGORY      VARCHAR2 (20) ,
    PRICE         FLOAT (126) ,
    SEASON        VARCHAR2 (20)
)
TABLESPACE DATA
LOGGING
;

CREATE UNIQUE INDEX ADMIN.PRODUCT_PK ON ADMIN.DIM_PRODUCT
(
    PRODUCT_ID ASC
)
TABLESPACE DATA
LOGGING
;

ALTER TABLE ADMIN.DIM_PRODUCT
ADD CONSTRAINT PRODUCT_PK PRIMARY KEY ( PRODUCT_ID )
USING INDEX ADMIN.PRODUCT_PK ;
```

Date Dimension Creation:

```
CREATE TABLE ADMIN.DIM_DATE
(
    DATE_ID    NUMBER GENERATED ALWAYS AS IDENTITY
                ( START WITH 1 CACHE 20 ) ,
    FULL_DATE  TIMESTAMP ,
    YEAR       VARCHAR2 (20) ,
    QUARTER    VARCHAR2 (20) ,
    MONTH      VARCHAR2 (20) ,
    WEEK       VARCHAR2 (20) ,
    DAY        VARCHAR2 (20)
)
TABLESPACE DATA
LOGGING
;

CREATE UNIQUE INDEX ADMIN.DATE_PK ON ADMIN.DIM_DATE
(
    DATE_ID ASC
)
TABLESPACE DATA
LOGGING
;

ALTER TABLE ADMIN.DIM_DATE
    ADD CONSTRAINT DATE_PK PRIMARY KEY ( DATE_ID )
    USING INDEX ADMIN.DATE_PK ;
```

Store dimension creation:

```
CREATE TABLE ADMIN.DIM_STORE
(
    STORE_ID          NUMBER GENERATED ALWAYS AS IDENTITY
                      ( START WITH 1 CACHE 20 ) ,
    STORE_LOCATION    VARCHAR2 (20) ,
    STORE_TYPE        VARCHAR2 (20) ,
    STORE_MANAGER     VARCHAR2 (20) ,
    REVENUE_TARGETS   VARCHAR2 (20)
)
TABLESPACE DATA
LOGGING
;
```

```
CREATE UNIQUE INDEX ADMIN.STORE_PK ON ADMIN.DIM_STORE
(
    STORE_ID ASC
)
TABLESPACE DATA
LOGGING
;
```

```
ALTER TABLE ADMIN.DIM_STORE
    ADD CONSTRAINT STORE_PK PRIMARY KEY ( STORE_ID )
    USING INDEX ADMIN.STORE_PK ;
```

Fact Sales creation:

```
CREATE TABLE ADMIN.FACT_SALES
(
    SALES_ID      NUMBER GENERATED ALWAYS AS IDENTITY
        ( START WITH 1 CACHE 20 ) ,
    QUANTITY      VARCHAR2 (20) ,
    TOTAL_PRICE   VARCHAR2 (20) ,
    PRODUCT_ID    NUMBER ,
    CUSTOMER_ID   NUMBER ,
    STORE_ID      NUMBER ,
    DATE_ID       NUMBER ,
    VENDOR_ID     NUMBER
)
TABLESPACE DATA
LOGGING
;

CREATE UNIQUE INDEX ADMIN.SALES_PK ON ADMIN.FACT_SALES
(
    SALES_ID ASC
)
TABLESPACE DATA
LOGGING
;

ALTER TABLE ADMIN.FACT_SALES
    ADD CONSTRAINT SALES_PK PRIMARY KEY ( SALES_ID )
    USING INDEX ADMIN.SALES_PK ;

ALTER TABLE ADMIN.FACT_SALES
    ADD CONSTRAINT SALES_TO_CUST FOREIGN KEY
    (
        CUSTOMER_ID
    ,
```

ETL PIPELINE DESIGN AND KEY COMPONENTS

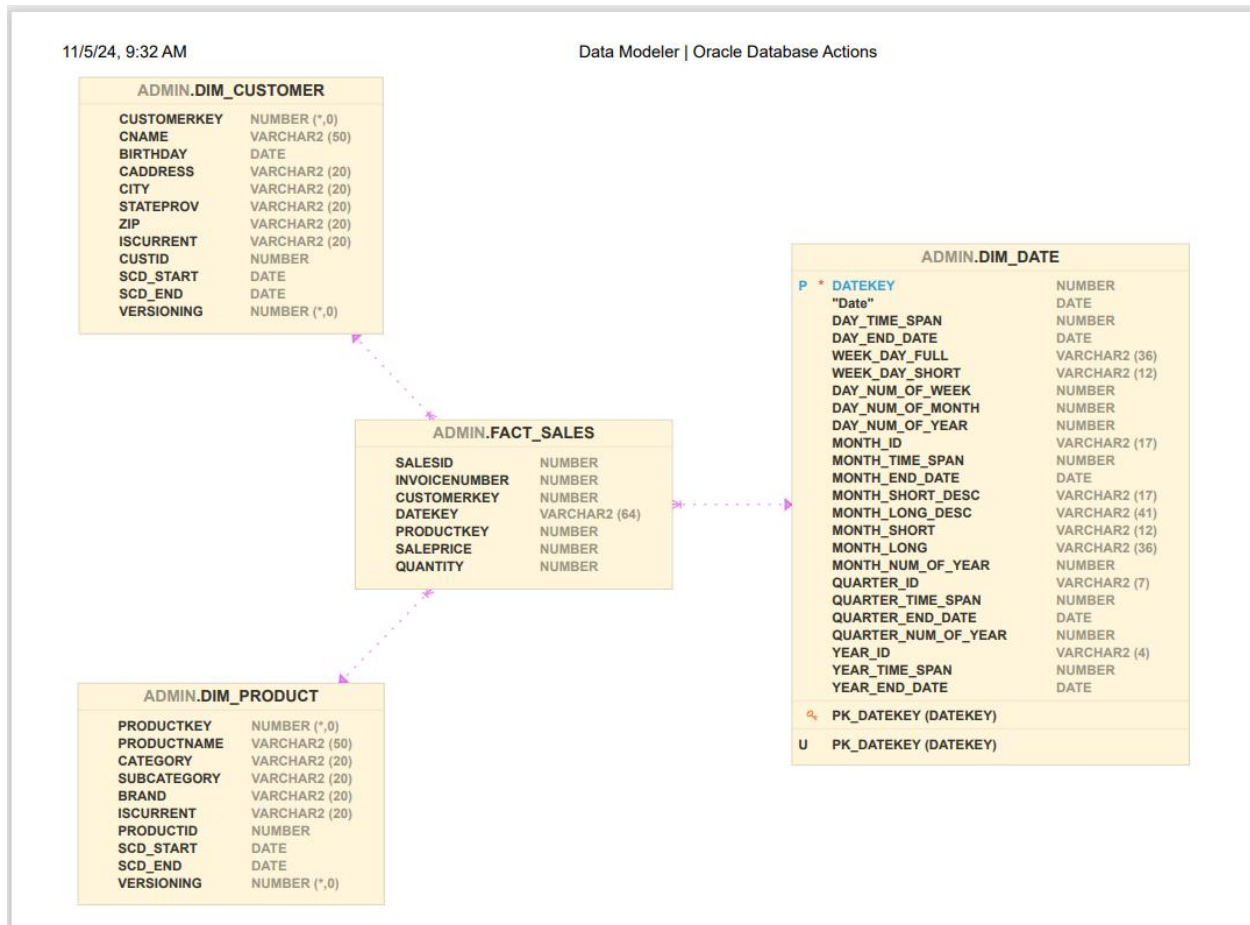
The ETL process in Apache HOP included three main phases:

1. **Data Extraction:** Data was sourced from CSV files and staged into Customer and Product dimension tables. These tables provided the primary datapoints to the FACT_SALES tables.
2. **Data Transformation:** One of the core transformation tasks was to handle the slowly changing dimensions for the DIM_PRODUCT and DIM_CUSTOMER tables, another challenge was to handle the null CUSTOMERKEY values. The pipeline was configured to set missing 'IsCurrent' values to 'Y' by default which resolved the issue.
3. **Data Loading:** The transformed data was then loaded into the FACT_SALES along with the associated dimensions into the Oracle Data Warehouse. The pipeline's adherence to a star schema architecture allowed for effective querying across several dimensions and offered an organized method for retrieving data.

TROUBLESHOOTING CHALLENGE AND SOLUTION:

Throughout the assignment, several technical challenges were encountered, and a significant issue arose with the **ORA-12838: cannot read/modify an object after modifying it in parallel error**. This error prevented updates to tables modified in parallel operations within the same session. Attempts to resolve the issue included creating a new warehouse and reconfiguring the ETL pipeline, but that did not help, and the solution ultimately involved setting the connection string to use the LOW service level, which disabled parallel processing. This workaround allowed the pipeline to operate in a single-threaded mode, effectively bypassing the error.

Database Diagram from Oracle Cloud Data Warehouse:



Based on the diagram generated, what is this database missing that you'd expect to see? Why might it be missing this component?

One component notably missing in the database diagram is a primary key in the **'FACT_SALES'** table. In a star schema, the fact table often has a primary key (or composite primary key) to uniquely identify each row. The absence of a primary key here might be due to the use of surrogate keys in the dimension tables, allowing the fact table to link to dimensions without enforcing a primary key constraint directly on it.

Additionally, foreign key constraints are also missing. Since these relationships will be managed within the ETL pipeline to maintain referential integrity and optimize performance, foreign key constraints are omitted from the data warehouse to reduce the load on it.

Loading the Dimension tables:

1. Output of SELECT * FROM DIM_DATE

	DATEKEY	DATE	DAY_TIME_SPAN	DAY_END_DATE	WEEK_DAY_FULL	WEEK_DAY_SHORT	DAY_NUM_OF_WEE	DAY_NUM_OF_MOI	DAY_NUM_OF_YEA	MONTH_ID	MONTH_TIME_SPAN
1	20180101	1/1/2018, 12:00:00 AM	1	1/1/2018, 12:00:00 AM	Monday	MON	2	1	1	JAN-2018	31
2	20180102	1/2/2018, 12:00:00 AM	1	1/2/2018, 12:00:00 AM	Tuesday	TUE	3	2	2	JAN-2018	31
3	20180103	1/3/2018, 12:00:00 AM	1	1/3/2018, 12:00:00 AM	Wednesday	WED	4	3	3	JAN-2018	31
4	20180104	1/4/2018, 12:00:00 AM	1	1/4/2018, 12:00:00 AM	Thursday	THU	5	4	4	JAN-2018	31
5	20180105	1/5/2018, 12:00:00 AM	1	1/5/2018, 12:00:00 AM	Friday	FRI	6	5	5	JAN-2018	31

	DI	MONTH_LONG_DE	MONTH_SHORT	MONTH_LONG	MONTH_NUM_OF	QUARTER_ID	QUARTER_TIME_SP	QUARTER_END_DA	QUARTER_NUM_OI	YEAR_ID	YEAR_TIME_SPAN	YEAR_END_DATE
1		January 2018	Jan	January	1	Q1-2018	800	3/19/2026, 12:00:00 AM	1	2018	365	12/31/2018, 12:00:00 AM
2		January 2018	Jan	January	1	Q1-2018	800	3/19/2026, 12:00:00 AM	1	2018	365	12/31/2018, 12:00:00 AM
3		January 2018	Jan	January	1	Q1-2018	800	3/19/2026, 12:00:00 AM	1	2018	365	12/31/2018, 12:00:00 AM
4		January 2018	Jan	January	1	Q1-2018	800	3/19/2026, 12:00:00 AM	1	2018	365	12/31/2018, 12:00:00 AM
5		January 2018	Jan	January	1	Q1-2018	800	3/19/2026, 12:00:00 AM	1	2018	365	12/31/2018, 12:00:00 AM

2. Output after updating the script

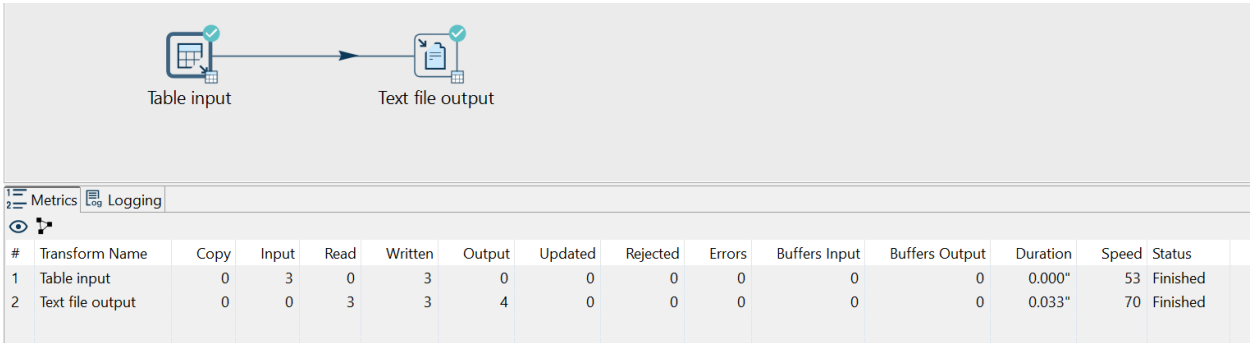
	DATEKEY	DATE	DAY_TIME_SPAN	DAY_END_DATE	WEEK_DAY_FULL	WEEK_DAY_SHORT	DAY_NUM_OF_WEE	DAY_NUM_OF_MOI	DAY_NUM_OF_YEA	MONTH_ID	MONTH_TIME_SPAN	M
1	20160102	1/2/2016, 12:00:00 AM	1	1/2/2016, 12:00:00 AM	Saturday	SAT	7	2	2	JAN-2016	31	1/
2	20160103	1/3/2016, 12:00:00 AM	1	1/3/2016, 12:00:00 AM	Sunday	SUN	1	3	3	JAN-2016	31	1/
3	20160104	1/4/2016, 12:00:00 AM	1	1/4/2016, 12:00:00 AM	Monday	MON	2	4	4	JAN-2016	31	1/
4	20160105	1/5/2016, 12:00:00 AM	1	1/5/2016, 12:00:00 AM	Tuesday	TUE	3	5	5	JAN-2016	31	1/
5	20160106	1/6/2016, 12:00:00 AM	1	1/6/2016, 12:00:00 AM	Wednesday	WED	4	6	6	JAN-2016	31	1/

	DI	MONTH_LONG_DE	MONTH_SHORT	MONTH_LONG	MONTH_NUM_OF	QUARTER_ID	QUARTER_TIME_SP	QUARTER_END_DA	QUARTER_NUM_OI	YEAR_ID	YEAR_TIME_SPAN	YEAR_END_DATE
1		January 2016	Jan	January	1	Q1-2016	993	1/1/2027, 12:00:00 AM	1	2016	365	12/31/2016, 12:00:00 AM
2		January 2016	Jan	January	1	Q1-2016	993	1/1/2027, 12:00:00 AM	1	2016	365	12/31/2016, 12:00:00 AM
3		January 2016	Jan	January	1	Q1-2016	993	1/1/2027, 12:00:00 AM	1	2016	365	12/31/2016, 12:00:00 AM
4		January 2016	Jan	January	1	Q1-2016	993	1/1/2027, 12:00:00 AM	1	2016	365	12/31/2016, 12:00:00 AM
5		January 2016	Jan	January	1	Q1-2016	993	1/1/2027, 12:00:00 AM	1	2016	365	12/31/2016, 12:00:00 AM

3. Output of SELECT * FROM DIM_PRODUCT (Initially I had not set the product key hence it shows NULL in the screenshot below, later I updated these NULL values while loading the FACT_SALES table.)

	PRODUCTKEY	PRODUCTNAME	CATEGORY	SUBCATEGORY	BRAND	ISCURRENT	PRODUCTID	SCD_START	SCD_END	VERSIONING
1	(null)	Cinnamon Bread	Wheat	Bread	Nothing Breader	Y	1	1/1/2024, 12:00:00 AM	12/31/2099, 12:00:00 AM	1
2	(null)	Milk	Dairy	Liquid	Buffalo Farms	Y	2	2/1/2024, 12:00:00 AM	12/31/2099, 12:00:00 AM	1
3	(null)	Chocolate Chip Cooki	Candy	Cookies	Nothing Breader	Y	3	3/1/2024, 12:00:00 AM	12/31/2099, 12:00:00 AM	1
4	(null)	Eggs	Dairy	Solid	Rochester Farms	Y	4	4/1/2024, 12:00:00 AM	12/31/2099, 12:00:00 AM	1
5	(null)	Rotini	Wheat	Pasta	Buffalo Farms	Y	5	5/1/2024, 12:00:00 AM	12/31/2099, 12:00:00 AM	1

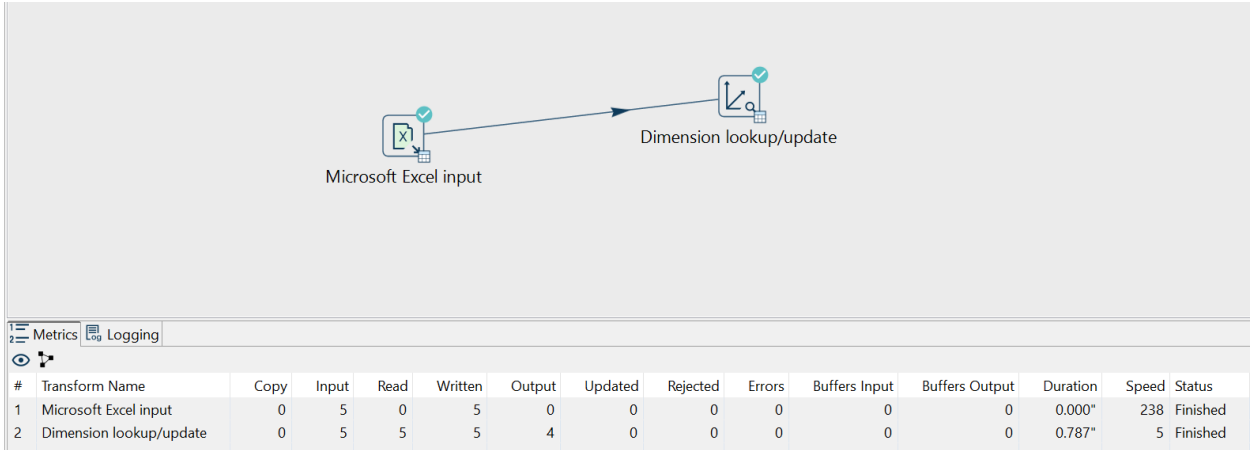
Text File pipeline screenshot



Output of SELECT * FROM DIM_PRODUCT in the Oracle data warehouse for QA check

ORACLE Database Actions											
	PRODUCTKEY	PRODUCTNAME	CATEGORY	SUBCATEGORY	BRAND	ISCURRENT	PRODUCTID	SCD_START	SCD_END	VERSIONING	
1	1	Cinnamon Bread	Wheat	Bread	Nothing Breadr	N	1	1/1/2024, 12:00:00 A	11/12/2024, 8:43:05 F	1	1
2	4	Milk	Dairy	Liquid	Buffalo Farms	Y	2	2/1/2024, 12:00:00 A	12/31/2099, 12:00:00	1	1
3	5	Rotini	Wheat	Pasta	Buffalo Farms	Y	5	5/1/2024, 12:00:00 A	12/31/2099, 12:00:00	1	1
4	3	Chocolate Chip Cooki	Candy	Cookies	Nothing Breadr	Y	3	3/1/2024, 12:00:00 A	12/31/2099, 12:00:00	1	1
5	0	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	1	1
6	103	Cinnamon Bread Loaf	Wheat	Bread	Nothing Breadr	Y	1	11/12/2024, 8:43:02 F	12/31/2199, 11:59:59	2	2
7	4	Eggs	Poultry	Solid	Rochester Farms	Y	4	11/12/2024, 8:43:02 F	12/31/2199, 11:59:59	2	2
8	105	Sugary Cereal	Wheat	Cereal	Food For You	Y	6	11/12/2024, 8:43:02 F	12/31/2199, 11:59:59	1	1
9	4	Eggs	Dairy	Solid	Rochester Farms	N	4	4/1/2024, 12:00:00 A	11/12/2024, 8:43:05 F	1	1

Screenshot of Apache Hop Pipeline.



Output of SELECT * FROM DIM_CUSTOMER

	CUSTOMERKEY	CNAME	BIRTHDAY	CADDRESS	CITY	STATEPROV	ZIP	ISCURRENT	CUSTID	SCD_START	SCD_END	VERSIONING
1	0	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)	1
2	1000	Dominic Sellitto	(null)	123 New St.	Rochester	NY	14321	(null)	1	11/12/2024, 8:52:01 F	12/31/2199, 11:59:59	2
3	1001	Jeep Jeeperson	(null)	123 Cool St.	Buffalo	NY	14043	(null)	2	11/12/2024, 8:52:01 F	12/31/2199, 11:59:59	2
4	1002	James Bond	(null)	543 Bond Rd.	Buffalo	NY	14222	(null)	4	11/12/2024, 8:52:01 F	12/31/2199, 11:59:59	1
5	1003	Jennifer Lopez	(null)	91 Perfect Ave.	Rochester	NY	14321	(null)	5	11/12/2024, 8:52:01 F	12/31/2199, 11:59:59	1
6	1	Dominic Sellitto	1/1/1956, 12:00:00 A	123 ABC St.	Buffalo	NY	14222	Y	1	12/31/2021, 12:00:00	11/12/2024, 8:52:03 F	1
7	2	Jeep Sellitto	2/2/1979, 12:00:00 A	123 Cool St.	Buffalo	NY	14222	Y	2	12/31/2021, 12:00:00	11/12/2024, 8:52:03 F	1
8	3	Sally Sallerson	3/3/1989, 12:00:00 A	415 Awesome Pl.	Rochester	NY	54321	Y	3	12/31/2021, 12:00:00	12/31/2099, 12:00:00	1

Loading Fact Table:

Stream Lookup Screenshot

Stream lookup

Transform name

Stream lookup 2

Lookup transform

DIM_CUSTOMER input

The key(s) to look up the value(s):

#

Field

Lookup field

1

CustID

CUSTID

Specify the fields to retrieve :

#

Field

New name

Default

Type

1

CUSTOMERKEY

None

Preserve memory (costs CPU) ☒

Key and value are exactly one integer field ☐

Use sorted list (i.s.o. hashtable) ☐

Help

OK

Cancel

Get key fields

Get return fields

Screenshot of Dim_Customer table input

Table input

Transform name: DIM_CUSTOMER input

Connection: New_low_OCIDW

SQL: `SELECT * FROM ADMIN.DIM_CUSTOMER WHERE ISCURRENT='Y'`

Line 1 Column 1

Replace variables in script? ☐

Insert data from transform

Execute for each row? ☐

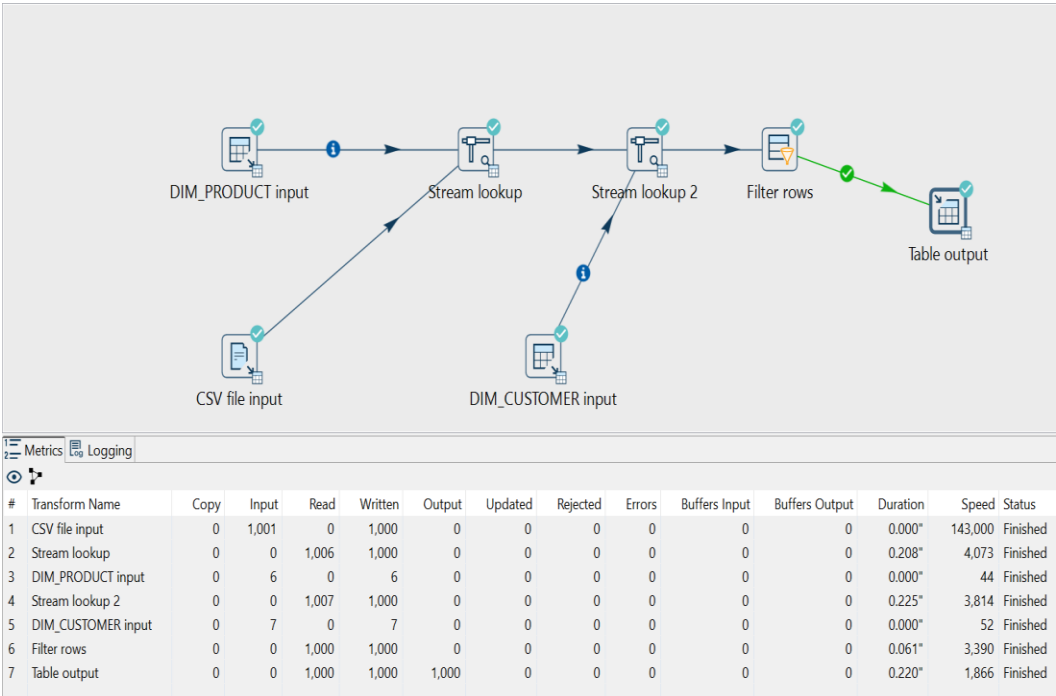
Limit size: 0

Buttons: Help, OK, Preview, Cancel

We're not doing a lookup for the date dimension. Write 1-2 sentences detailing why we don't need to.

We are not performing a lookup for the date dimension because the DATEKEY in the date dimension does not directly match the date format in the fact CSV. Since the date values in the fact CSV do not align with the structure of the DATEKEY, a direct lookup is unnecessary and instead it requires a transformation to align the date formats before joining.

Final Screenshot of the completed pipeline and the select query output.



ORACLE Database Actions							
	SALESID	INVOICENUMBER	CUSTOMERKEY	DATEKEY	PRODUCTKEY	SALEPRICE	QUANTITY
982	982	982	3	6272021	103	11	1
983	983	983	3	6292021	4	20	3
984	984	984	2	6292021	4	5	4
985	985	985	3	6302021	103	7	2
986	986	986	4	7012021	103	22	3
987	987	987	5	7022021	3	6	4
988	988	988	2	7072021	5	23	5
989	989	989	1	7072021	105	2	1
990	990	990	4	7092021	103	7	1
991	991	991	2	7102021	4	28	5
992	992	992	4	7112021	105	17	4
993	993	993	3	7112021	3	13	2
994	994	994	1	7122021	5	6	5
995	995	995	2	7142021	4	1	3
996	996	996	4	7152021	5	14	2
997	997	997	5	7152021	4	29	2
998	998	998	5	7152021	4	4	4
999	999	999	4	7162021	5	27	3
1000	1000	1000	2	7172021	5	26	5

Finally, Power BI Dashboard Creation

The primary objective of this assignment was to utilize a visualization tool to forecast and analyze sales data of TGS, transforming raw information into actionable insights. I chose Power BI for this purpose. The project involved installing Power BI Desktop, preparing and cleaning the data, developing interactive dashboards, and applying forecasting techniques to provide a comprehensive understanding of sales trends and performance metrics.

Actions Taken:

1. Installation of Power BI Desktop and Data Import:

The first step was downloading and installing Power BI Desktop. After installation, it was connected to the Oracle data warehouse to import customer data, product data, date dimension data, and sales data. This rich dataset served as the foundation for analysis and dashboard creation, allowing for a detailed examination of sales performance from various perspectives.

2. Data Preparation and Relationship Management:

The imported data from the warehouse was cleaned and formatted to maintain consistency and accuracy, with data types adjusted as needed. Relationships between tables, such as product table and sales records, were established to enable seamless data integration across various visualizations. This process ensured that the data model could support advanced analytics and interactive filtering.

3. Creating Visualizations:

Three primary visualizations were designed to extract and present key insights:

(a) Sum of Sale Price by Category (Pie Chart):

A pie chart was created to illustrate the proportion of total sales contributed by each product category. The chart revealed that Wheat products accounted for the largest share (37.23%) of total sales, followed by Dairy and Poultry, each contributing 24.55%. Candy represented the smallest share at 13.68%. This visualization provided a clear breakdown of sales distribution across categories, highlighting the most and least dominant segments.

Recommendation: To capitalize on high demand, TGS can add new varieties or bundles to popular wheat products. Depending on consumer preferences, either

improve marketing for underperforming candy products or swap them out for more in-demand items.

(b) Sum of Sale Price by Product Name (Bar Chart):

A bar chart was developed to showcase the total sales for individual products. The chart indicated that Eggs and Milk were the top-selling products, each generating significant revenue. In contrast, products such as Chocolate Chip Cookies, Rotini, Cinnamon Bread Loaf, and Sugary Cereal had comparatively lower sales. This visualization allowed for easy comparison of product performance, helping to identify top-performing items and potential areas for improvement.

Recommendation: To increase sales, we can use targeted promotions and loyalty benefits to highlight popular products like milk and eggs. Also, examine underperforming items, such as rotini and chocolate chip cookies, and adjust or replace them in response to consumer input.

(c) Sum of Sale Price by Year (Line Chart with Forecast):

A line chart was created to visualize sales trends from 2018 to 2021, showing steady growth from 2018 to 2020, followed by a slight dip in 2021. To offer forward-looking insights, a forecasting feature was applied to project sales for 2023, with a 95% confidence interval shading to represent the potential range of future sales. This forecasting tool provided valuable insights into future performance based on historical trends, aiding in strategic planning.

Recommendation: Considering TGS's goal to expand, the company should prioritize maximizing profitability and optimizing operations in current stores before expanding, given the forecast's modest growth. It should also experiment with new strategies, such as pilot stores or online channels, to evaluate their potential success before committing to large-scale expansion.

4. Dashboard Design and Storytelling:

The final step involved designing a cohesive dashboard that effectively conveyed the insights derived from the data. The layout was organized to ensure clarity and ease of interpretation, with each visualization playing a distinct role in telling the overall sales story. The above combination of pie charts, bar graphs, and line charts created

a comprehensive view of sales performance, highlighting key trends and areas for focus.

Dashboard narrative:

The Power BI dashboard which is attached as a PDF to the assignment offers a thorough analysis of GUAC STOP's sales performance together with insightful information about important business domains. The distribution of total sales across the various product categories is shown in the pie chart, with wheat as the best-performing product, followed by dairy and poultry. This highlights the necessity of concentrating on growing these highly sought-after categories. By delving deeper into certain products, the bar chart shows that milk and eggs are the best-selling items. These might be the focus of promotions to increase recurring business. A dynamic look at sales trends over time is provided by the line chart with forecast, which displays consistent increase from 2018 to 2020, a minor decline in 2021, and a projected 2023 growth.

Before expanding, significant thought should be given to the forecast's uncertainty, which is shown by the confidence interval. This dashboard enables GUAC STOP to make data-driven choices that optimize product offers, maximize profitability, and assess novel approaches such as pilot stores or online channels to get ready for future expansion.

